# 07 · Multicore-Echtzeit und Shared Memory

AMP, Speicherbesitz, Cache-Sicht und MPU · EZS · DHBW Stuttgart

Prof. Dr.-Ing. Daniel Klünder

**Folie** 1 / 50

### 07 · Zwei Kerne, ein verlässlicher Datenvertrag

- Das Ein-Kern-Modell auf zwei Kerne erweitern
- Vom lokalen Kernel zum Austausch zwischen Firmwares
- Zuständigkeit, Speicherbesitz und Sichtbarkeit
- Übung: Papiermodelle und gemeinsamer Speicher

**Notizen** 1 / 50

Vorlesung 6 hat zusätzliche Ereignisarbeit auf einem Kern durch Serverbudgets begrenzt. Dieser Termin erweitert nun das Ein-Kern-Bild: Was ändert sich, wenn zwei Kerne eigene Programme ausführen und trotzdem Hardware teilen? Multicore bedeutet mehrere Prozessorkerne in einem System. Auf dem STM32H755 des Boards NUCLEO-H755ZI-Q betrachten wir den Cortex-M7 und den Cortex-M4. Bislang haben Threads desselben FreeRTOS-Kernels Ressourcen geschützt, Nachrichten ausgetauscht und um begrenzte Prozessorzeit konkurriert. Jetzt kommunizieren zwei getrennte Firmwareprogramme. Firmware ist das für den Mikrocontroller übersetzte und geladene Programm.
Das gemeinsame Datenobjekt liegt im Arbeitsspeicher. Damit sind die bisherigen Fragen nach Kapazität, Lebensdauer und Schreibrecht weiterhin relevant. Hinzu kommt, ob ein Zugriff wirklich den gemeinsamen Speicher erreicht oder nur eine lokale Cachekopie. Ein Cache ist ein schneller Zwischenspeicher eines Kerns. Wir bauen diese Begriffe Schritt für Schritt auf und verwenden ein kleines Ping-Pong als durchgängiges Beispiel. Der Cortex-M4 verwendet weiterhin FreeRTOS, ein Echtzeitbetriebssystem; der Cortex-M7 arbeitet Bare Metal, also ohne diesen Kernel. Das Beispiel liefert einen überprüfbaren Datenvertrag, aber noch keinen allgemeinen Nachweis jeder Multicore-Deadline.

**Folie** 2 / 50

## Vorlesungen 1–6 liefern die Voraussetzungen

| Bekannt | Erweiterung heute |
|---|---|
| Deadline und Antwortzeit | Interferenz durch gemeinsame Hardware |
| Ready und Blocked | zwei getrennte Ausführungskontexte |
| Mutex und Semaphor | Geltungsbereich prüfen |
| Queue und Pufferbesitz | Übergabe zwischen zwei Images |
| Serverbudget und Überlast | Kapazität je Kern und gemeinsame Störung |

**Notizen** 2 / 50

Eine Deadline ist die Abschlussfrist einer geforderten Reaktion. Die Antwortzeit reicht von der Freigabe bis zur Fertigstellung. Interferenz bedeutet zusätzliche Verzögerung durch konkurrierende Arbeit. Ready heißt ausführungsbereit, Blocked auf eine Bedingung oder Zeit wartend. Ein Mutex ist eine Ressourcensperre mit Besitzer, ein Semaphor ein begrenzter Zähler von Berechtigungen oder Arbeitshinweisen. Eine Queue ist eine begrenzte Nachrichtenwarteschlange.
Vorlesung 5 hat gezeigt, dass eine erfolgreiche Kopie nicht automatisch eine abgeschlossene Verarbeitung und ein gespeicherter Pointer nicht automatisch einen verfügbaren Puffer bedeutet. Vorlesung 6 hat Queueinhalt, eigene Prozessorzeit und zeitliche Bedienung getrennt. Ein Serverbudget gilt dabei für das analysierte Ausführungsmodell; ein zweiter Kern erweitert es nicht automatisch um seine freie Kapazität. Heute kommt eine weitere Grenze hinzu: Die Kernelverwaltung gehört zu einer bestimmten Firmware. Image bezeichnet deren übersetztes Programm mit eigenem Speicherlayout. Ein M4-Queue-Handle wird durch Ablegen in gemeinsamem Speicher nicht zum Objekt des M7. Kenntnisse über C-Funktionen, Strukturen und die bisherige Cube-Projektbasis genügen für den Einstieg. Neue Hardwarebegriffe werden im jeweiligen Zusammenhang eingeführt; ein Betriebssystem für mehrere Kerne wird nicht als bereits bekannt vorausgesetzt.

**Folie** 3 / 50

## Was Sie am Ende begründen können

1. SMP und AMP samt gemeinsamen Störquellen unterscheiden
2. Adresse, Schreibrecht und Sichtbarkeit getrennt prüfen
3. Das Schreibrecht eines Ping-Pong-Protokolls erklären
4. Barriere, Cachewartung und MPU auseinanderhalten
5. Einen vorbereiteten Basislauf begründet auswerten

**Notizen** 3 / 50

SMP bedeutet Symmetric Multiprocessing, symmetrische Mehrkernverarbeitung mit einer gemeinsamen Betriebssystemverwaltung. AMP bedeutet Asymmetric Multiprocessing, asymmetrische Mehrkernverar-

**Notizen, Fortsetzung** 3 / 50

beitung mit getrennten Softwarekontexten. Ein Linker verbindet übersetzte Programmteile und legt ihre Speicheradressen fest. Das Linkerlayout bestimmt deshalb, ob beide Firmwareprogramme tatsächlich denselben physischen Speicher benutzen.
Das Schreibrecht beantworten wir über einen Protokollzustand, also eine vereinbarte Phase des Datenaustauschs. Eine Barriere ordnet Speicherzugriffe. Cachewartung behandelt lokale Zwischenspeicherkopien. Die MPU, Memory Protection Unit oder Speicherschutzeinheit, legt Rechte und Attribute für Regionen eines Kerns fest. Diese drei Mittel lösen unterschiedliche Probleme und dürfen nicht als austauschbare Befehle verstanden werden. Der Basisversuch nutzt absichtlich eine nicht cachebare gemeinsame Region und genau einen Teilnehmer pro Kern. Die Lehrperson bereitet Boot, Linker und Speicherattribute vor. Sie prüfen den daraus folgenden Vertrag und implementieren einen kleinen Antwortschritt selbst. Ein gesonderter Cachefehlerbuild ist eine optionale Vertiefung. Die Lernziele werden mit Papieraufgaben und nachvollziehbaren Prüfstellen überprüft, nicht allein durch die Beobachtung eines erfolgreichen Builds.

**Folie** 4 / 50

## Vier Schulstunden mit vorbereiteter Projektbasis

| Block | Ablauf in Minuten |
|---|---|
| 1 · Kerne und Zuständigkeiten | 25 Erklärung + 15 Papier 1 + 5 Projektbasis |
| 2 · Datenvertrag und Protokoll | 30 Erklärung + 15 Papier 2 |
| 3 · Sichtbarkeit und Grenzen | 20 Erklärung + 20 Papier 3–4 + 5 Integration |
| 4 · Versuch und Rückblick | 25 Versuch + 15 Quiz + 5 Abschluss |

**Notizen** 4 / 50

Vier Schulstunden entsprechen hier 180 Minuten Unterricht, ohne zusätzliche Pausen. Die Tabelle umfasst 75 Minuten Erklärung, 50 Minuten Papierarbeit, 35 Minuten Projektprüfung und Versuch sowie 20 Minuten Quiz und Abschluss. Die etwa 50 Folien sind Arbeitsstationen. Eine Rechen- oder Integrationsfolie bleibt deutlich länger sichtbar als eine kurze Orientierung.
Die Erklärung konzentriert sich auf Architektur, Schreibrecht, Sichtbarkeit und Interferenz. Vorgegebene Adressen, Größen und Attribute werden auf Papier geprüft; die vollständige Einrichtung der Plattform steht in der Implementierungshilfe. Der Programmierteil setzt zwei bereits lauffähige Images der Projektbasis voraus. Image bedeutet das übersetzte Programm für einen Kern. Die Lehrperson prüft vorab Boot, Linkerdateien, Zeitbasen und Debugstart und stellt die Anwendungsdateien bereit. Eine erstmalige Installation, die Reparatur des Dual-Core-Starts und die Suche nach einem falsch geflashten Ziel passen nicht zusätzlich in die 35 Minuten. Ohne passende Hardware können die Papieraufgaben vollständig bearbeitet und der Code gemeinsam erklärt werden. Fehlende Boardmessungen bleiben offen. Der Aufbau vermeidet außerdem Aufgaben, die noch nicht eingeführte Cache- oder MPU-Kenntnisse voraussetzen: Die entsprechenden Papierfragen liegen nach der Erklärung dieser Begriffe.

**Folie** 5 / 50

## Zwei Kerne machen Überlast nicht automatisch lösbar

Lehrmodell mit fester Zuordnung:

$$U_{M4} = 1{,}10, \qquad U_{M7} = 0{,}20$$

- Gesamte Pflichtlast liegt unter zwei Kernkapazitäten
- Auf M4 fehlt trotzdem dauerhaft Prozessorzeit
- M7 übernimmt lokale M4-Aufgaben nicht automatisch
- Eine neue Aufteilung braucht einen neuen Daten- und Zeitvertrag

**Notizen** 5 / 50

U bezeichnet die langfristige Pflichtauslastung eines Kerns, also die Summe aus eigenem Jobbedarf geteilt durch Freigabeperiode. Die Zahlen sind Lehrannahmen, keine Messung: Auf dem Cortex-M4 werden pro Zeiteinheit 110 Prozent seiner Kapazität benötigt, auf dem Cortex-M7 zwanzig Prozent. Zusammen sind das 1,30 Kernkapazitäten und damit weniger als zwei. Trotzdem ist der M4 dauerhaft überfordert, wenn die Zuordnung fest bleibt.
AMP, Asymmetric Multiprocessing oder getrennte Mehrkernsoftware, migriert eine Task nicht selbstständig zum anderen Kern. Der M7 kann den M4 erst unterstützen, wenn eine Aufgabe ausdrücklich ausgelagert und ihre Ein- und Ausgabe übertragen wird. Das kann neue Warte-, Kopier- und Buskosten erzeugen. Auch unter einem anderen Mehrkern-Scheduler wäre die Gesamtauslastung allein keine universelle Fristgarantie für jede Auswahlregel. Der bekannte Ein-Prozessor-Test aus Vorlesung 2 wird daher nicht einfach durch zwei ersetzt. Wir unterscheiden zuerst die Softwarearchitekturen, bevor wir über eine sinnvolle Aufteilung der Arbeit entscheiden.

**Folie** 6 / 50

## Arbeit aufteilen heißt Verantwortlichkeiten festlegen

| Cortex-M4 | Cortex-M7 |
|---|---|
| zeitkritische Erfassung und Regelung | aufwendige Auswertung oder Darstellung |
| eigener FreeRTOS-Kernel | eigene Bare-Metal-Struktur |
| veröffentlichter Messdatensatz | daraus abgeleitete Antwort |

Aufteilung ist ein Entwurf, keine Garantie durch den Kernnamen.

**Notizen** 6 / 50

Die Tabelle zeigt eine mögliche Aufgabenverteilung, keine feste Eigenschaft des Chips. Erfassung bedeutet Datenaufnahme, Regelung eine zeitgebundene Reaktion auf diese Daten. Darstellung oder aufwendige

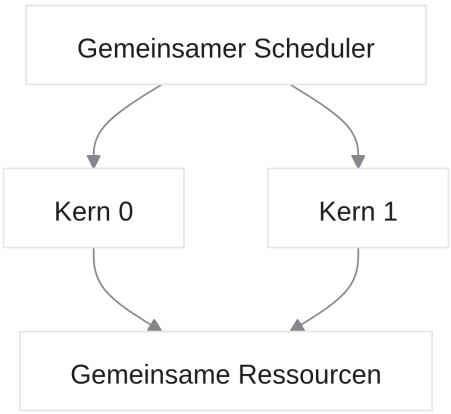
**Notizen, Fortsetzung** 6 / 50

Auswertung kann separat organisiert werden. FreeRTOS ist das Echtzeitbetriebssystem des M4; Bare Metal bezeichnet hier das Programm des M7 ohne diesen Kernel.
Die Wahl kann Fristnachweise übersichtlicher machen, wenn jeder Kern begrenzte Aufgaben und klare Datenübergaben erhält. Der Cortex-M4 ist dadurch aber nicht automatisch störungsfrei und der Cortex-M7 nicht automatisch für weiche Fristen reserviert. Beide können gemeinsame Speicherwege und Peripherie beanspruchen. Auch ein ausgelagerter Arbeitsschritt kann die Gesamtantwort verzögern, wenn sein Ergebnis gebraucht wird. Aus Vorlesung 5 übernehmen wir deshalb die Idee eines Datensatzes mit einem Vertrag. Wer erzeugt ihn, wer konsumiert ihn und wann darf sein Speicher wieder beschrieben werden? Die folgenden Architekturformen beantworten zunächst, wer die Aufgaben auswählt und welche Softwareverwaltung gemeinsam ist.

**Folie** 7 / 50

## SMP: ein gemeinsamer Scheduler, mehrere Kerne

Gemeinsame Verwaltung kann Aufgaben auf geeignete Kerne zuteilen.



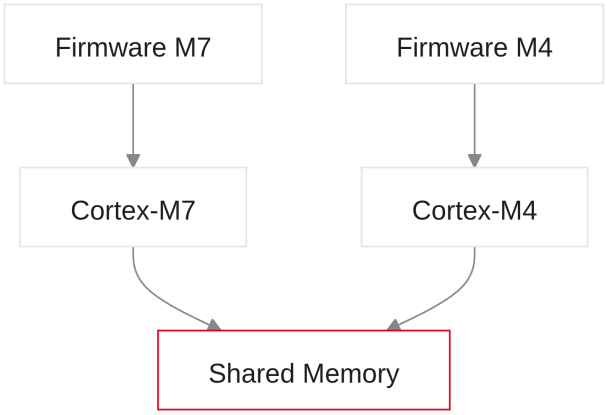

**Notizen** 7 / 50

SMP steht für Symmetric Multiprocessing, symmetrische Mehrkernverarbeitung. Ein Scheduler wählt ausführungsbereite Aufgaben und teilt sie geeigneten Kernen zu. Die Abbildung wird von oben nach unten gelesen: Eine gemeinsame Verwaltung erreicht zwei Kerne, die anschließend gemeinsame Hardware benutzen können. Die grauen Pfeile beschreiben Zuständigkeiten und Zugriffe, keine maßstäbliche Zeitachse.
Migration bedeutet, dass eine Aufgabe auf einem anderen Kern weiterlaufen kann. Ob das erlaubt ist, hängt vom Scheduler und von der Affinität ab. Affinität beschränkt die zulässigen Kerne einer Aufgabe. Mehrere Kerne brauchen trotzdem nicht identische Ausführungszeiten oder einen gemeinsamen L1-Cache. L1 bezeichnet die erste Cacheebene. Die Zeichnung ist ein allgemeines Architekturmodell und keine Behauptung, dass das heutige STM32-Projekt einen solchen Kernel verwendet. Ein geeigneter SMP-Entwurf kann Echtzeitfristen erfüllen; die Analyse muss Migration, gemeinsame Ressourcen und seine konkrete Auswahlregel berücksichtigen. Für den Kursversuch wählen wir eine andere, ausdrücklich getrennte Verwaltung.

**Folie** 8 / 50

## AMP: getrennte Firmwares, gemeinsame Hardware

Unser Labor: M7 Bare Metal, M4 FreeRTOS.




**Notizen** 8 / 50

AMP bedeutet Asymmetric Multiprocessing, asymmetrische Mehrkernverarbeitung. Die Abbildung zeigt zwei Firmwareprogramme mit getrennten Ausführungskontexten. Firmware M7 läuft auf dem Cortex-M7, Firmware M4 auf dem Cortex-M4. Die Pfeile zum gemeinsamen Speicher stehen für Zugriffe; die rote Umrandung hebt den späteren Datenkanal hervor. Shared Memory ist Speicher, den beide erreichen können.

Im Labor verwendet der M4 FreeRTOS als Echtzeitkernel und der M7 ein Bare-Metal-Programm ohne diesen Kernel. Andere AMP-Systeme können andere Betriebssysteme kombinieren. Ein allgemeines Beispiel mit Linux auf einem größeren Cortex-A wird nicht auf den H755 übertragen. Getrennte Firmware bedeutet getrennte lokale Aufgabenverwaltung und keine automatische Migration zwischen diesen Kernen. Sie bedeutet aber keine vollständige Hardwareisolation. Der gemeinsame Speicher, Buswege und Taktkonfiguration bleiben mögliche Kopplungen. Der Vorteil für den Einstieg ist die klare Zuständigkeit: Der Datenvertrag kann zwischen genau zwei Beteiligten beschrieben werden. Sein Erfolg hängt anschließend von Adressen, Schreibrecht und Sichtbarkeit ab.

**Folie** 9 / 50

## Softwaretrennung ersetzt kein Interferenzmodell

- Private Rechenarbeit und gemeinsame Buslast wirken verschieden
- Affinität begrenzt Migration, nicht alle Speicherzugriffe
- Getrennte Kernel entfernen gemeinsame Ressourcen nicht
- Timing braucht begründete Budgets und Störgrenzen

„M7 ist ausgelastet“ beschreibt die Art der Last noch nicht.

**Notizen** 9 / 50

Interferenz bedeutet hier Verzögerung durch konkurrierende Hardwarezugriffe oder andere relevante Arbeit. Affinität ist die Einschränkung einer Aufgabe auf bestimmte Kerne. Migration ist deren Wechsel zu einem anderen Kern. Beide Begriffe stammen aus der Mehrkern-Aufgabenverwaltung. Ein privater Rechenpfad kann weniger gemeinsame Ressourcen benutzen als eine Schleife, die ständig auf denselben Speicher wie die Gegenstelle zugreift.
TCM bedeutet Tightly Coupled Memory, eng an einen Kern gekoppelter Speicher. Wenn der M7 Code und Daten in geeigneten lokalen Bereichen benutzt und keine weiteren gemeinsamen Zugriffe erzeugt, entsteht eine andere Störlast als bei vielen SRAM4-Zugriffen. SRAM ist Static Random Access Memory, statischer Arbeitsspeicher. Das ist eine qualitative Entwurfsbeobachtung, kein gemessener Zahlenfaktor. Interrupts und zusätzliche Hardwaremaster können die Voraussetzung verändern. Eine WCET, Worst-Case Execution Time oder obere Schranke der eigenen Ausführungszeit, muss die relevanten Störungen passend einschließen. Der Prozessorname und seine Auslastungsanzeige liefern diese Schranke nicht allein.

**Folie** 10 / 50

## Ein lokales Kernelobjekt bleibt lokal

| Bekanntes Mittel | Geltungsbereich im Labor |
|---|---|
| M4-Queue-Handle | Verwaltung im M4-Kernel |
| M4-Mutex | kooperierende Teilnehmer dieses Kernels |
| lokale IRQ-Maskierung | Interrupts des aufrufenden Kerns |
| Shared-Block mit Protokoll | ausdrücklich vereinbarte Gegenstelle |

Ein Handle im SRAM wird dadurch kein gemeinsamer Scheduler.

**Notizen** 10 / 50

Ein Handle ist ein Bezeichner eines von einer Softwareverwaltung angelegten Objekts. Die Queue, begrenzte Nachrichtenwarteschlange, und der Mutex, Ressourcensperre mit Besitzer, gehören hier zum FreeRTOS-Kernel des Cortex-M4. Sein M7-Gegenüber führt diesen Kernel nicht aus und nimmt deshalb nicht automatisch an dessen Wartelisten, Speicherverwaltung oder Sperrregel teil.
IRQ bedeutet Interrupt Request, Interruptanforderung. Eine lokale Interruptmaskierung verhindert bestimmte Behandler auf dem Kern, der sie ausführt. Sie hält den anderen Kern und DMA, Direct Memory Access oder direkten Hardware-Speicherzugriff, nicht automatisch an. Ein M4-Queue-Handle im gemeinsamen SRAM, Static Random Access Memory oder Arbeitsspeicher, ersetzt folglich keinen kernübergreifenden Kanal. Der Kanal benötigt ein gemeinsames Datenlayout und ein Protokoll. Dieses unterscheidet die vorhandenen Bytes von der Verwaltungssoftware, die auf sie zugreift. Die Tabelle knüpft direkt an Vorlesung 4 und 5 an: Ein Schutzmittel wirkt nur für die Beteiligten und Zugriffe, die seinen Vertrag tatsächlich einhalten.

**Folie** 11 / 50

## Zwei verschiedene Kerne auf dem STM32H755

| Eigenschaft | Cortex-M7 | Cortex-M4 |
|---|---|---|
| spezifiziertes Maximum | bis 480 MHz | bis 240 MHz |
| L1-Daten-/Instruktionscache | vorhanden | kein M7-artiger L1-Cache |
| eng gekoppelter Speicher | ITCM/DTCM | kein direkter gemeinsamer TCM-Kanal |
| Laborsoftware | Bare Metal | FreeRTOS |

Tatsächliche Takte und Bedingungen im Projekt prüfen.

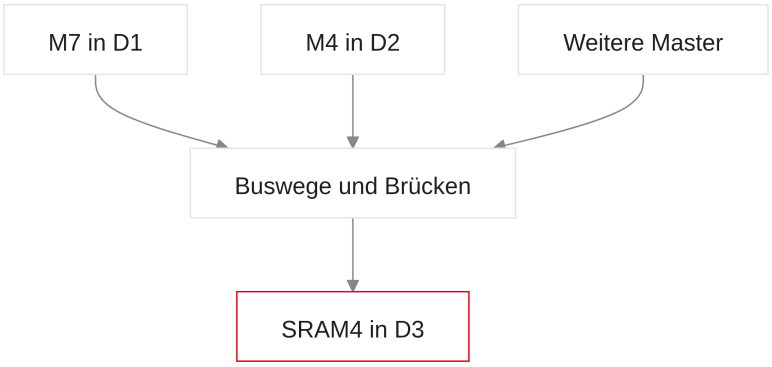
**Notizen** 11 / 50

MHz bedeutet Megahertz, Millionen Takte je Sekunde. Die angegebenen Werte sind spezifizierte Maxima unter passenden Betriebsbedingungen, nicht die im Kurs bereits eingestellten Takte. L1 ist die erste Cacheebene. ITCM und DTCM bedeuten Instruction und Data Tightly Coupled Memory, eng gekoppelten Befehls- und Datenspeicher des M7. Sie sind kein geeigneter direkter gemeinsamer Arbeitsplatz für M4-Zugriffe.
Der M4 hat auf diesem Chip keinen M7-artigen L1-Daten- und Instruktionscache. Sein Flashpfad kann dennoch Beschleunigung und Wartezyklen enthalten. ART steht für Adaptive Real-Time Accelerator, einen ST-Flashbeschleuniger. Auch ohne diesen L1-Cache bleiben Buskonflikte, Speicherwartezeiten und Interrupts relevant. Bare Metal bedeutet hier M7-Code ohne Echtzeitkernel; FreeRTOS ist der Kernel auf dem M4. Das passt zur Softwarearchitektur AMP, Asymmetric Multiprocessing. Für Timing und Speicherwahl zählen die konkrete Taktkonfiguration, die erreichbare Region und deren Attribute. Eine pauschale Zusage gleicher Zugriffszeit oder nanosekundengenauer Ausführung lässt sich aus der Tabelle nicht ableiten.

**Folie** 12 / 50

## Domänen und gemeinsame Zugriffswege

Vereinfachte Auswahl: D1 mit M7, D2 mit M4, SRAM4 in D3.




**Notizen** 12 / 50

D1, D2 und D3 sind Domänenbezeichnungen des STM32H755, keine Threadzustände. Die Abbildung zeigt eine vereinfachte Auswahl der Zugriffswege und wird von oben nach unten gelesen. Der M7 ist D1, der M4 D2 zugeordnet; SRAM4 liegt in D3. SRAM bedeutet Static Random Access Memory, Arbeitsspeicher. Eine Brücke verbindet Busbereiche, ein Master initiiert einen Zugriff. Die rote Umrandung markiert unser gemeinsames Speicherziel.
AHB bedeutet Advanced High-performance Bus und AXI Advanced eXtensible Interface; beides sind Busfamilien. Weitere Master können beispielsweise DMA-Einheiten sein, Direct Memory Access oder direkter Hardware-Speicherzugriff. Die konkrete Instanz und deren erreichbare Ziele werden im Referenzhandbuch geprüft. Die Zeichnung ist weder eine vollständige Peripheriekarte noch eine Messung. Sie trennt Domänenzuordnung, Zugriffsweg und Speicherziel. Ein Low-Power-Modus kann die Erreichbarkeit oder den Erhalt einer Region verändern. Im Basisversuch bleiben die benötigten Domänen aktiv. Zwei verschiedene Kerne können trotzdem an derselben endlichen Zielressource konkurrieren.

**Folie** 13 / 50

## Getrennte Rechenkerne, trotzdem ein gemeinsames Budget

Lehrannahme: nicht überlappende Antwortanteile.

$$R = C_{M4} + W_{IPC} + I_{Bus}$$

$$R = 4 + 3 + 2 = 9\,\text{ms} > D = 8\,\text{ms}$$

- eigene M4-Arbeit: 4 ms
- Warten auf das benötigte Ergebnis: 3 ms
- zusätzliche Businterferenz: 2 ms

**Notizen** 13 / 50

R ist die Antwortzeit bis zum geforderten Abschluss, D die relative Deadline ab Freigabe. C M4 bezeichnet vier Millisekunden eigener Arbeit auf dem Cortex-M4. W IPC bezeichnet drei Millisekunden Warten auf ein benötigtes Ergebnis der Inter-Processor Communication, Kommunikation zwischen Prozessorkernen. I Bus ist eine zusätzlich angesetzte Verzögerung von zwei Millisekunden durch Buskonkurrenz. ms sind Millisekunden.
Die Formel ist ein vereinfachtes Lehrbudget, keine allgemeine Multicore-Analysemethode. Die drei Anteile sind hier ausdrücklich getrennt und nicht überlappend. Ihre Summe beträgt neun Millisekunden und überschreitet die Frist acht um eine Millisekunde. Wenn eine gemessene Wartezeit bereits die Buskonkurrenz enthält, darf diese nicht nochmals addiert werden. Ebenso kann ein anderes Protokoll Teile parallel ausführen. Das Beispiel zeigt die praktische Bedeutung der Störgrenze: Ein eigener M4-Bedarf von vier Millisekunden reicht als Nachweis nicht, wenn die Reaktion noch ein fremdes Ergebnis braucht. Papieraufgabe 1 prüft diese Unterscheidung gemeinsam mit den Architekturformen.

**Folie** 14 / 50

### Takte und Peripherie brauchen feste Zuständigkeiten

- M7 initialisiert die gemeinsamen Takte im gewählten Bootmuster
- Pin Context bestimmt die Codegenerierung
- Zwei Pins eines Ports können dasselbe Konfigurationsregister teilen
- Eine lokale MPU ist kein Schloss für den gesamten Chip

Zuständigkeit folgt dem Register oder Gerät, nicht nur dem Pinnamen.

**Notizen** 14 / 50

RCC steht für Reset and Clock Control, die Takt- und Rücksetzungssteuerung des Chips. Im gewählten Startmuster konfiguriert der M7 die gemeinsamen Takte, bevor der M4 seinen normalen Anwendungsstart fortsetzt. Zwei unkoordinierte SystemClock_Config-Aufrufe können dieselben Register widersprüchlich ändern.
Pin Context ist die CubeMX-Zuordnung für die Codegenerierung. Sie ist keine Hardware-Zugriffssperre. GPIO bedeutet General Purpose Input/Output, allgemeiner Ein- und Ausgang. Verschiedene Pins eines GPIO-Ports können Felder desselben Konfigurationsregisters belegen. Lesen beide Kerne dessen alten Stand, ändern verschiedene Felder und schreiben zurück, kann ein Lost Update entstehen: Eine Änderung geht verloren. Ein atomarer Set-/Reset-Ausgangspfad ist ein anderer Vorgang und löst nicht automatisch die Konfigurationsfrage. MPU bedeutet Memory Protection Unit, Speicherschutzeinheit eines Kerns. Sie koordiniert nicht beide Programme oder jeden Hardwaremaster. Ein klarer Besitzer für gemeinsam zu konfigurierende Geräte und Register ist deshalb bereits vor dem ersten Datenaustausch erforderlich.

**Folie** 15 / 50

## Boot: Stop, Reset und laufendes Programm unterscheiden

Vorbereitete Basis: Startfolge am eigenen Projekt bestätigen:

1. M4 richtet HSEM-Benachrichtigung ein und geht in Stop.
2. M7 wartet darauf und initialisiert gemeinsame Takte.
3. M7 richtet den Shared-Block ein.
4. HSEM-Freigabe lässt M4 weiterlaufen und den Kernel starten.

Option Bytes können die Startvoraussetzungen verändern.

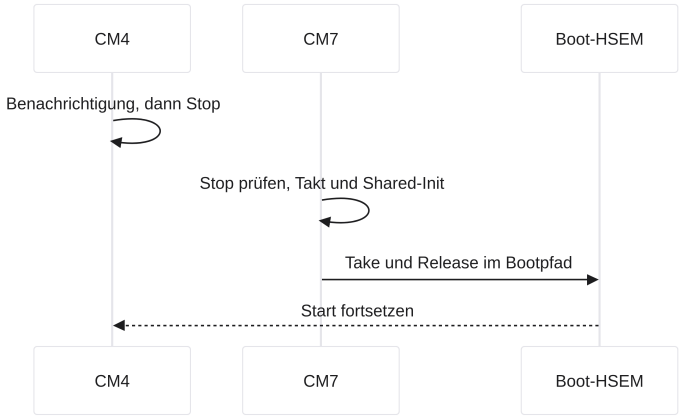
**Notizen** 15 / 50

Boot bezeichnet den Startablauf nach dem Rücksetzen. HSEM bedeutet Hardware Semaphore, Hardwaresemaphor. Er kann eine Freigabe beziehungsweise Benachrichtigung zwischen den Kernen unterstützen. Im typischen Cube-Muster läuft auf dem M4 bereits Startcode, bevor er in Stop geht. Stop ist ein Energiespar-Wartezustand und nicht gleichbedeutend mit gehaltenem Reset. Nach der vereinbarten Benachrichtigung setzt der M4 seinen Start fort.
Option Bytes sind nichtflüchtige Konfigurationswerte, die unter anderem die Startvoraussetzungen beeinflussen. Sie werden am konkreten Projekt und Gerät geprüft. Der M7 wartet mit dem Muster des erzeugten Codes auf den M4-Zustand, richtet Takte und unseren Block ein und gibt erst danach frei. Seine frühe Schleifengrenze ist nicht automatisch eine gemessene Millisekundenfrist. HSEM-Identität, Taktfreigabe und Benachrichtigung müssen auf beiden Seiten zusammenpassen. Wir behalten den funktionierenden erzeugten Handshake, also den abgestimmten Startvertrag. Die konkrete Einrichtung ist Teil der Projektvorbereitung; für die Aufgabe begründen Sie vor allem, warum der Datenblock vor seiner Nutzung gültig sein muss. Eine aus Erinnerung ergänzte, im installierten Header nicht vorhandene Wake-Funktion wäre kein verlässlicher Ersatz.

**Folie** 16 / 50

## Vor der Freigabe muss der Block gültig sein

Logische Reihenfolge, keine Zeitmessung.



**Notizen** 16 / 50

CM4 und CM7 bezeichnen die Cortex-M4- und Cortex-M7-Projektteile. HSEM ist der Hardware Semaphore, hier ausschließlich der Boot-Hinweis. Die Pfeile des Sequenzbilds zeigen Funktions- und Zuständigkeitsreihenfolge von oben nach unten; es gibt keine skalierte Zeitachse. Der M4 bereitet die Benachrichtigung vor und geht in Stop. Der M7 bestätigt diese Phase, richtet gemeinsame Takte und den Datenblock ein und führt den vorhandenen Take-/Release-Pfad zur Freigabe aus.
Take ist der Erwerb des Hardwareobjekts, Release dessen Freigabe. Die im Template verwendete Identität wird nicht aus einer allgemeinen Folie geraten. Nach der Benachrichtigung muss der M4 noch seinen lokalen Start und den FreeRTOS-Kernelstart durchlaufen. Deshalb ist eine erste Anwendungsantwort nicht identisch mit der Bootfreigabe. Das öffentlich eingesehene ST-Beispiel zeigt das grundsätzliche Muster; das H755-Projekt ist weiterhin die verbindliche Integrationsbasis. Init nach der Freigabe wäre zu spät: Die M4-Anwendung könnte bereits lesen. Auch ein späteres erneutes Löschen durch den M4 würde den gemeinsamen Vertrag verletzen.

**Folie** 17 / 50

## Zwei Images haben zwei Speicherlayouts

- Jedes Kernprogramm hat sein eigenes Speicherlayout.
- Ein gemeinsamer Header vereinbart Typ und Bedeutung.
- Beide Maps müssen dieselben physischen Shared-Bytes zeigen.
- Private Daten dürfen die gemeinsame Reserve nicht belegen.

Boot- und Buildkonfiguration gehören zur vorbereiteten Basis.

**Notizen** 17 / 50

Image bezeichnet das übersetzte Programm eines Kerns. Eine Linkerdatei legt seine Speicherplatzierung fest; die Map ist das daraus erzeugte Platzierungsprotokoll. Ein Header ist eine gemeinsam verwendete Deklarationsdatei. Zwei Programme können denselben Typ und Symbolnamen benutzen und trotzdem unterschiedliche physische Adressen ansprechen. Gemeinsamer Datenzugriff entsteht deshalb nicht allein durch denselben C-Header.
Jedes Image hat außerdem einen eigenen Startup, also frühen Startcode, und eine eigene Vektortabelle mit Behandlereinstiegen. ELF bedeutet Executable and Linkable Format, das Programmformat mit Code, Daten und Adressinformationen. Diese Dateien und Startadressen werden vor dem Versuch geprüft. STM32CubeMX erzeugt die Projektbasis; Visual Studio Code mit STM32 VS Code Extension baut ihre CMake-Ziele. CMake beschreibt Übersetzung und Verknüpfung. Die konkreten Integrationsschritte stehen in der Implementierungshilfe. Hier ist die überprüfbare Kernaussage entscheidend: Die tatsächlich geladenen beiden Programme müssen dieselben Shared-Bytes reservieren, und kein privater Heap oder Stack darf diesen Bereich zusätzlich benutzen. Heap ist dynamisch verwalteter Speicher, Stack der Stapelspeicher für Aufrufe und lokale Daten.

**Folie** 18 / 50

## SRAM4 ist die bewusste Laborentscheidung

| Speicher | Bedeutung für den Kanal |
|---|---|
| M7-ITCM/DTCM | kein direkter gemeinsamer M4-Arbeitsplatz |
| andere erreichbare SRAM-Bänke | prinzipiell nach Zugriffsprüfung möglich |
| SRAM4 in D3 | gewählter Bereich, 64 KiB |

Adresse: `0x38000000` bis einschließlich `0x3800FFFF`.

**Notizen** 18 / 50

SRAM bedeutet Static Random Access Memory, statischer Arbeitsspeicher. SRAM4 liegt beim betrachteten STM32H755 im Bereich von 0x38000000 bis einschließlich 0x3800FFFF. Die Länge beträgt 65536 Byte oder 64 KiB; KiB steht für Kibibyte mit 1024 Byte. D3 ist die zugehörige Domänenbezeichnung.
ITCM und DTCM bedeuten Instruction und Data Tightly Coupled Memory, eng gekoppelten Befehls- und Datenspeicher des M7. Sie sind kein geeigneter direkter gemeinsamer M4-Arbeitsplatz. Andere Speicherbänke können je nach Zugriffsweg ebenfalls für Kommunikation geeignet sein. Dafür müssen Kern- und Hardwaremaster-Erreichbarkeit, Takt- und Energiestatus sowie die Speicherattribute geprüft werden. SRAM4 ist deshalb eine Laborentscheidung und keine Vorschrift für jede Anwendung. Im Einstieg reservieren beide Images diese Bank ausdrücklich für den Kanal. Private Stacks, dynamischer Speicher und zusätzliche unkoordinierte Puffer bleiben außerhalb. Die physische Erreichbarkeit beantwortet nur, ob die Kerne die Region adressieren können; sie beantwortet noch nicht deren genaue Platzierung oder Cache-Sicht.

**Folie** 19 / 50

## Derselbe Name ist noch nicht dieselbe Adresse

| Prüfung | CM7-Map | CM4-Map |
|---|---|---|
| Symbol `ipc` | `0x38000000` | `0x38000000` |
| Typgröße | 12 Byte | 12 Byte |
| reservierte Sektion | 32 Byte | 32 Byte |

Lehrsoll, keine bereits gemessene oder gelinkte Board-Map.

**Notizen** 19 / 50

Eine Map ist das Platzierungsprotokoll des Linkers. CM7 und CM4 sind die getrennten Kernziele. ipc ist der vereinbarte Name unseres gemeinsamen Datenobjekts. Die Tabelle zeigt das verlangte Ergebnis, nicht eine im Repository vorliegende Firmware-Map. Der Linker kann gleichnamige Variablen zweier

**Notizen, Fortsetzung** 19 / 50

unabhängiger Programme an verschiedene Adressen legen. In diesem Fall würden beide korrekt auf ihre eigene Stelle zugreifen und trotzdem nicht kommunizieren.
Beide Maps müssen ipc bei 0x38000000 zeigen. Der Typ hat zwölf Byte, die gepolsterte Sektion reserviert eine 32-Byte-Zeile. Diese Größen sind nicht gleichbedeutend. Die gleichzeitige Reservierung in zwei Images erzeugt keine zwei physischen SRAM-Blöcke: Beide Adressen bezeichnen hier dieselben Bytes. Umgekehrt reicht derselbe Symbolname in zwei Debugfenstern nicht als Beweis. Zusätzlich werden andere Sektionen, Heap und Stack auf Überlappung geprüft. Heap ist der dynamisch verwaltete Speicher. Erst der konkrete Linker- und Startup-Vertrag verhindert, dass ein vermeintlich privater Bereich die gemeinsamen Bytes überschreibt.

**Folie** 20 / 50

## Die Reservierung ist Teil des Datenvertrags

| Prüfpunkt | Muss für beide Images gelten |
|---|---|
| Shared-Beginn | dieselbe physische Adresse |
| Reserve | Objekt, Ausrichtung und Polster passen |
| Private Daten | kein überlappender Heap oder Stack |
| Startcode | kein späteres Löschen gemeinsamer Daten |

Die Linkereinrichtung steht in der Implementierungshilfe.

**Notizen** 20 / 50

Ein Linker verbindet übersetzte Programmteile und legt ihre Adressen fest. Ein Image ist das übersetzte Kernprogramm. Shared bezeichnet den gemeinsam benutzten Speicher. Die Tabelle ist die Prüfliste für die vorbereitete Plattform: gleicher Beginn, ausreichende gemeinsame Reserve, kein überlappender privater Speicher und eine dazu passende Initialisierung. Heap ist dynamischer Speicher, Stack der Aufruf- und lokale Datenspeicher. Ein korrekter Zahlenlauf allein belegt diese Bedingungen nicht.
Für unseren Basisblock stehen zwölf Byte Typinhalt in einer auf 32 Byte ausgerichteten und gepolsterten Sektion. Der gewählte Ursprung bleibt 0x38000000. Der vollständige Linker-Integrationsausschnitt steht in der Projektanleitung und im Fragment shared-section.ld. Seine Prüfungen sichern Adresse, Objektinhalt und Reserve ab. Eine Map protokolliert das Ergebnis des tatsächlich verwendeten Linkerskripts. Zusätzlich werden die Startup-Tabellen gelesen, also die Kopier- und Nulltabellen des frühen Starts. Reservierung bedeutet hier eine Abmachung beider Programme über dieselben physischen Bytes, keine zweite Hardwarekopie und noch keinen gegenseitigen Ausschluss. Die folgende Typdefinition beschreibt danach, welche Informationen diese Bytes enthalten.

**Folie** 21 / 50

### Drei Wörter und ein gemeinsamer Header

```c
typedef struct {
  uint32_t magic;
  uint32_t state;
  uint32_t payload;
} ipc_block_t;
extern volatile ipc_block_t ipc;
```

Offsets: 0, 4, 8 · Typ: 12 Byte · Objekt: auf 32 Byte ausgerichtet.
Genau eine Definition je Image, keine Definition im Header.

**Notizen** 21 / 50

Der C-Code ist der Typ- und Deklarationsausschnitt des vollständigen Moduls. uint32_t ist ein vorzeichenloser 32-Bit-Typ, hier vier Byte. magic enthält die vereinbarte Protokollkennung, state die Phase und payload den Nutzwert. ipc_block_t ist der Typname, ipc das konkrete Objekt. extern deklariert das Objekt, reserviert es aber nicht erneut in jeder einbindenden Datei. volatile fordert die vom Compiler definierten sichtbaren Zugriffe; es ist noch kein Cachevertrag.
Die vollständige Referenz prüft Größe und Feldoffsets mit statischen C-Assertions. Die Definition in lab07_shared.c wird genau einmal pro Image übersetzt. Ein GCC-Attribut legt das Objekt in .shared und richtet es auf 32 Byte aus. GCC bedeutet GNU Compiler Collection, die Compilerfamilie. Die natürliche Typgröße bleibt zwölf Byte. Ausrichtung des Objekts, gepolsterte Sektionsgröße und Größe der MPU-Region sind drei Ebenen. MPU steht für Memory Protection Unit, Speicherschutzeinheit. Der gemeinsame Header enthält außerdem Konstanten und Funktionsdeklarationen, damit beide Firmwareprogramme denselben Datenvertrag verwenden.

**Folie** 22 / 50

### NOLOAD schützt nicht vor jedem Startup-Schreibzugriff

- `.data`: üblicherweise Kopie initialer Werte
- `.bss`: üblicherweise Nullinitialisierung
- `.shared`: keine automatische zweite Initialisierung
- M7 initialisiert einmal, bevor M4 freigegeben wird
- Gemeinsamer Neustart gehört zum Basisvertrag

Ein gültiges Magic ist noch kein Nachweis einer neuen Sitzung.

**Notizen** 22 / 50

Startup ist der frühe Startcode eines Images. .data enthält gewöhnlich globale Daten mit Initialwerten, die in den Laufzeitspeicher kopiert werden. .bss bezeichnet typischerweise globale Nullinitialisierung. .shared ist

**Notizen, Fortsetzung** 22 / 50

unsere gemeinsame Sektion. NOLOAD ist eine Linkerkennzeichnung ohne normale Initialisierungsdaten, keine allgemeine Anweisung an jede beliebige Startschleife oder jeden Debugloader.
Beide Startup-Tabellen müssen deshalb die gemeinsamen Bytes aus ihrer privaten Kopier- und Nullbehandlung ausschließen. Nach Reset ist der Inhalt noch kein gültiger Protokollzustand. Der Cortex-M7 setzt ihn einmal vor der vereinbarten M4-Freigabe; der Cortex-M4 löscht ihn nach dem Aufwachen nicht. Magic ist eine vereinbarte Kennung. Ein alter gültiger Wert beweist keine neue Sitzung nach einseitigem Core-Reset. Der Basisversuch benutzt einen gemeinsamen Neustart und unterstützt keine automatische Wiederaufnahme mitten in einer Runde. Diese Grenze verhindert die gefährliche Schlussfolgerung, bei Timeout einfach denselben Block zu nullen, während die Gegenstelle noch schreiben könnte.

**Folie** 23 / 50

## Drei getrennte Fragen an jede Übergabe

| Frage | Mittel im Basisversuch |
|---|---|
| Wer darf schreiben? | Phasenbesitz des Protokolls |
| Was kommt zuerst? | Datenzugriffe und Barriere |
| Welche Bytes sieht der andere? | gleiche Adresse, nicht cachebare Region |

Keines dieser Mittel ersetzt die beiden anderen.

**Notizen** 23 / 50

Phasenbesitz bedeutet, dass genau ein Teilnehmer in einer bestimmten Kommunikationsphase den Nutzwert verändern darf. Das ist dieselbe Grundidee wie der Pufferbesitz aus Vorlesung 5, hier zwischen zwei Firmwares. Eine Barriere ist ein Prozessorbefehl, der die Reihenfolge relevanter Speicherzugriffe absichert. Eine nicht cachebare Region verhindert für diesen Kern die Verwendung einer entsprechenden privaten Datenkopie.
Eine korrekte Adresse verhindert noch keinen gleichzeitigen Schreibkonflikt. Eine Besitzregel verhindert noch keine veraltete Cachekopie. Ein einzelner Barrierebefehl schreibt eine schmutzige Cachezeile nicht automatisch in den gemeinsamen Speicher zurück. Auch ein perfekter Cachevertrag ersetzt die Phasenfolge nicht. Diese Trennung ist der rote Faden der zweiten Hälfte. Wir führen zunächst das einfache Besitzprotokoll ein. Danach ordnen wir die Zugriffe und erklären zuletzt, warum dieselben korrekt ausgeführten Loads und Stores trotzdem verschiedene Werte liefern können. Die Tabelle ist ein Entwurfsraster für den Basisversuch und keine universelle Synchronisationsbibliothek für beliebige Prozessorarchitekturen.

**Folie** 24 / 50

## Ping-Pong: der Zustand übergibt das Schreibrecht

Genau ein Teilnehmer je Kern; drei Antworten, dann Abschluss.




**Notizen** 24 / 50

Die Abbildung zeigt einen Kommunikationszustandsautomaten, keine Threadzustände oder Präemptionszeitachse. TURN_M7 bedeutet, dass der M7 die Initialisierung beziehungsweise den abschließenden Zustand besitzt. REQUEST_M4 ist eine veröffentlichte Anfrage an den M4. RESPONSE_M7 ist dessen veröffentlichte Antwort an den M7. Die Pfeile stehen für erlaubte Protokollübergänge. M7 und M4 bezeichnen Cortex-M7 und Cortex-M4.
Vor der ersten Freigabe legt der M7 die Anfrage an. Bei REQUEST_M4 darf der M4 den Nutzwert lesen und ändern. Bei RESPONSE_M7 darf der M7 die Antwort lesen und gegebenenfalls die nächste Anfrage schreiben. Der vorige Besitzer verändert den Nutzwert nach seiner Übergabe nicht weiter. Im vollständigen Basiscode werden drei Antworten bearbeitet. Danach veröffentlicht der M7 TURN_M7, ohne einen vierten Request zu erzeugen; der M4-Worker beendet sich nach seiner dritten Antwort. Das endliche Ende ist eine Laborentscheidung. Ein späterer dauerhafter Kanal bräuchte einen erweiterten Betriebs- und Fehlervertrag. Der Zustand allein funktioniert nur zusammen mit der folgenden Zugriffsreihenfolge und Sichtbarkeit.

**Folie** 25 / 50

## Aus null werden die Antworten eins, drei und fünf

| Aktion | Payload | Zustand danach |
|---|---|---|
| M7 initialisiert | 0 | REQUEST_M4 |
| M4 antwortet | 1 | RESPONSE_M7 |
| M7 nächste Anfrage | 2 | REQUEST_M4 |
| M4 antwortet | 3 | RESPONSE_M7 |
| M7 nächste Anfrage | 4 | REQUEST_M4 |
| M4 antwortet | 5 | RESPONSE_M7 |
| M7 beendet | 5 | TURN_M7 |

**Notizen** 25 / 50

Payload ist der eigentliche Nutzwert. REQUEST_M4 und RESPONSE_M7 sind die vereinbarten Anfrage- und Antwortphasen; TURN_M7 ist der M7-eigene Abschlusszustand. Die Tabelle ist eine logische Schrittfolge, keine Messung oder Zeitachse. M7 und M4 stehen für die beiden Cortex-Kerne.

**Notizen, Fortsetzung** 25 / 50

Der M7 legt anfangs null ab. Der M4 erhöht diesen Wert auf eins und veröffentlicht die Antwort. Der M7 kopiert eins in seine private Diagnose, erhöht den gemeinsamen Wert auf zwei und veröffentlicht die nächste Anfrage. Daraus werden drei, dann vier und schließlich fünf. Die privaten M7-Antwortwerte sind somit eins, drei und fünf. Null ist der Startwert und keine erste Antwort. Nach der dritten Antwort wird kein vierter Request mehr erzeugt. Die gemeinsamen Bytes behalten den Nutzwert fünf. Ein Read-Modify-Write, Lesen, Ändern und Zurückschreiben, findet hier nur durch den jeweiligen Phasenbesitzer statt. Der Vorgang ist keine allgemein atomare Mehrkern-Inkrementfunktion. Papieraufgabe 2 fordert deshalb ausdrücklich Zustand und Besitzer zusätzlich zu den Zahlen.

**Folie** 26 / 50

## Veröffentlichen: erst Nutzdaten, dann Zustand

Protokollausschnitt im berechtigten M7:

```
ipc.payload = next_value;
__DMB();
ipc.state = REQUEST_M4;
```

Die Gegenstelle darf das veröffentlichte Signal erst für die fertigen Daten sehen.
Voraussetzungen: passender Besitz und vereinbarte Speicherattribute.

**Notizen** 26 / 50

Der C-Ausschnitt zeigt die Veröffentlichung, nicht die vollständige Initialisierung oder Fehlerbehandlung. ipc ist das gemeinsame volatile-Objekt, payload dessen Nutzwert und next_value die bereits bereitgestellte nächste Anfrage. REQUEST_M4 ist der vereinbarte Zustand für den Cortex-M4. Die vollständige Modulreferenz kennt next_value hier als den um eins erhöhten Antwortwert.
DMB bedeutet Data Memory Barrier, Datenspeicherbarriere. __DMB ist die Cortex-M-Schnittstelle zum entsprechenden Befehl. Sie ordnet die relevanten Datenzugriffe vor der nachfolgenden Zustandsveröffentlichung. Der M4 soll nicht bereits die neue Phase verarbeiten, während die dazugehörigen Nutzdaten noch nicht in der richtigen Reihenfolge veröffentlicht sind. Das setzt den bekannten Phasenbesitz und passende Speicherattribute voraus. Es ist kein atomarer Store der gesamten Struktur und keine automatische Rückschreibung eines Write-Back-Caches. Write Back bedeutet Rückschreiben geänderter Cachekopien zu einem späteren Zeitpunkt. Im nicht cachebaren Basisversuch reicht die Protokollfolge ohne manuelle Cachewartung; der Leser braucht trotzdem den passenden Gegenpart.

**Folie** 27 / 50

### Konsumieren: Zustand prüfen, dann Daten lesen

Protokollausschnitt im M7 nach gültiger Kennung:

```
if (ipc.state == RESPONSE_M7) {
  __DMB();
  uint32_t reply = ipc.payload;
  /* reply gehört zu dieser Antwortphase. */
}
```

Bei falschem Zustand wird die Payload nicht als neue Antwort verarbeitet.

**Notizen** 27 / 50

Der C-Ausschnitt zeigt das Lesen einer Antwort. ipc ist das gemeinsame Objekt, RESPONSE_M7 der erwartete Antwortzustand, payload der Nutzwert und reply eine private Kopie im Cortex-M7. uint32_t ist ein vorzeichenloser 32-Bit-Typ. Der vollständige Code hat zuvor die Kennung magic geprüft und behandelt falsche Zustände oder Zeitablauf gesondert.
DMB, Data Memory Barrier oder Datenspeicherbarriere, steht nach dem beobachteten Zustand und vor dem zugehörigen Nutzdatenlesen. Die Gegenrichtung auf dem Cortex-M4 folgt derselben Idee: REQUEST_M4 prüfen, Barriere, Anfrage lesen. Nur der bestätigte Phasenwechsel erlaubt die Interpretation als neue Nachricht. Ein früher gelesener payload-Wert ist keine alternative Erfolgsprüfung. Die private Kopie entkoppelt nach dem Lesen die Diagnose von späterer Wiederverwendung des gemeinsamen Nutzwerts. Dieser Lesepfad braucht trotzdem eine gültige Speicheransicht. Wenn der M7 den Zustand aus einer alten Cachekopie liest, erzwingt DMB nicht dessen Aktualisierung. Reihenfolge und Cache-Sicht sind daher weiterhin getrennte Fragen.

**Folie** 28 / 50

### volatile beeinflusst den Compiler, nicht den Cache

- Eine Pollschleife muss den vorgesehenen Zugriff erneut ausführen
- Der Load kann trotzdem aus dem D-Cache kommen
- `payload++` bleibt Lesen, Ändern und Schreiben
- Der ganze Struct wird nicht auf einmal veröffentlicht

Das Basisprotokoll ist eine konkrete Armv7-M-/Toolchain-Vereinbarung.

**Notizen** 28 / 50

volatile ist ein C-Typqualifikator für Zugriffe, deren Wirkung nicht wie eine gewöhnliche private Variable wegoptimiert werden darf. Ein Load ist das Laden eines Werts. D-Cache bezeichnet den Datenzwischenspeicher des Kerns. Ein wieder ausgeführter Load kann von diesem Cache beantwortet werden, statt die aktuellen gemeinsamen SRAM-Bytes zu lesen. SRAM bedeutet Static Random Access Memory,

**Notizen, Fortsetzung** 28 / 50

Arbeitsspeicher.
payload++ ist ein Read-Modify-Write, Lesen, Ändern und Schreiben. volatile macht daraus weder eine allgemeine atomare Mehrkernoperation noch die Veröffentlichung aller Strukturfelder in einem Schritt. Der Basisvertrag verwendet ausgerichtete uint32_t-Wörter, vorzeichenlose 32-Bit-Werte, passende Normal-Speicherattribute und die Armv7-M-Barrieren der eingebundenen Compiler-/Prozessorunterstützung. Armv7-M ist die Architektur der betrachteten Cortex-M4/M7-Kerne. Diese explizite Plattformvereinbarung ist keine portable ISO-C-Thread-Synchronisationsgarantie. ISO C bezeichnet die standardisierte C-Sprache. Es bleiben drei getrennte Nachweise: Was der Compiler ausführt, was die Hardware ordnet und ob die Gegenstelle aktuelle Daten sieht.

**Folie** 29 / 50

## DMB und DSB reinigen keine Cachezeile

| Mittel | Aufgabe | keine automatische Wirkung |
|---|---|---|
| DMB | relevante Datenzugriffe ordnen | Dirty-Cache zurückschreiben |
| DSB | Abschluss relevanter früherer Operationen abwarten | alle Cachekopien erneuern |
| ISB | Instruktionsausführung synchronisieren | fremden Kern sperren |

Die MPU-Konfiguration nutzt andere Barrieren als die reine Nachricht.

**Notizen** 29 / 50

DMB steht für Data Memory Barrier, Datenspeicherbarriere. DSB steht für Data Synchronization Barrier, Datensynchronisationsbarriere. ISB steht für Instruction Synchronization Barrier, Instruktionssynchronisationsbarriere. Eine Dirty-Cachezeile ist eine lokal geänderte Kopie, deren Änderung noch nicht zurückgeschrieben sein muss. Die Tabelle trennt die Hauptrollen; deren genauer Geltungsbereich folgt Architektur und Speicherattributen.
DMB ordnet relevante Zugriffe. DSB wartet auf den Abschluss relevanter früherer Operationen. Daraus folgt nicht, dass jede schmutzige Datenkopie nun im gemeinsamen Speicher steht: Die Cachewartungsoperation selbst muss vorgesehen sein. ISB synchronisiert die nachfolgende Instruktionsausführung, etwa im Zusammenhang mit geänderten MPU-Einstellungen. MPU bedeutet Memory Protection Unit, Speicherschutzeinheit. Keine dieser Barrieren sperrt automatisch den anderen Kern. Für unser nicht cachebares Ping-Pong stehen DMB an Veröffentlichung und Konsum. Der MPU-Helfer enthält die für seine Konfigurationsfolge nötigen Abschluss- und Instruktionsbarrieren. Beliebige zusätzliche Barrieren sind kein Ersatz für die fehlende Cache- oder Besitzregel.

**Folie** 30 / 50

## Boot-HSEM, Semaphor und Nutzdaten trennen

| Element | Rolle heute |
|---|---|
| Boot-HSEM | erzeugten Startvertrag fortsetzen |
| RTOS-Semaphor | lokales Kernelobjekt aus Vorlesung 4 |
| SRAM4-Block | Kennung, Phase und Nutzwert |
| Phasenprotokoll | Schreibrecht während der Runden |

HSEM-Laufzeitsperren bleiben eine optionale Vertiefung im Code-Archiv.

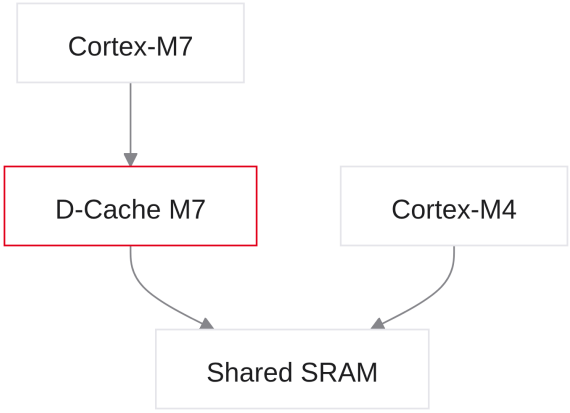
**Notizen** 30 / 50

HSEM bedeutet Hardware Semaphore, Hardwaresemaphor. Im Basisprojekt verwendet ihn der erzeugte Bootpfad, um den Cortex-M4 seine Initialisierung fortsetzen zu lassen. RTOS bedeutet Real-Time Operating System, Echtzeitbetriebssystem. Sein lokaler Semaphor verwaltet Token, also Arbeitshinweise oder Berechtigungen, innerhalb dieses Kernels. Beide Begriffe enthalten Semaphore, bezeichnen aber unterschiedliche Verwaltungen und Anwendungsrollen.
SRAM4 ist die ausgewählte gemeinsame Arbeitsspeicherbank und trägt die tatsächlichen Bytes. Der HSEM enthält weder unseren payload-Wert noch automatisch eine Nachrichtenwarteschlange. Im Laufzeit-Ping-Pong entscheidet die vereinbarte Phase über das Schreibrecht, nicht ein neues HSEM-Take vor jedem Wort. Ein M4-Mutex, Besitzsperre des lokalen Kernels, schützt den M7 nicht ohne dessen Teilnahme. Wenn später beliebige gleichzeitige Schreiber koordiniert werden müssen, kommt eine ausdrückliche kernübergreifende Sperrregel hinzu. Sie ersetzt weiterhin keine Barriere und keinen Cachevertrag. Heute bleiben genau ein Anwendungsteilnehmer pro Kern und ein klarer Besitzer pro Phase die bewusst überschaubare Voraussetzung.

**Folie** 31 / 50

## Ein Cache hält lokale Kopien von Speicherzeilen

Der M4 kann den privaten M7-D-Cache nicht direkt auslesen.




**Notizen** 31 / 50

Ein Cache ist ein schneller lokaler Zwischenspeicher. D-Cache bedeutet Daten-Cache. Eine Cachezeile ist die Einheit, in der ein Bereich kopiert und verwaltet wird; beim betrachteten Cortex-M7 sind das 32 Byte. Die Abbildung zeigt die Zugriffspfade von oben nach unten. Die rote Umrandung hebt die private M7-Kopie hervor. Die Pfeile stehen für Speicherzugriffe und nicht für zeitgleiche Verarbeitung oder eine Bandbreitenmessung.
Shared SRAM ist gemeinsam erreichbarer Static Random Access Memory, Arbeitsspeicher. Der M4 greift nicht über die private M7-D-Cachekopie auf diesen Bereich zu. Deshalb können beide Kerne dieselbe Adresse verwenden und trotzdem verschiedene Werte beobachten. Cachekohärenz bedeutet, dass diese Kopien mit dem gültigen gemeinsamen Datenstand zusammenpassen. Das H755-System stellt zwischen M7 und M4 keinen solchen automatischen L1-Datenabgleich bereit. Die Abbildung behauptet keinen gemeinsamen Cache beider Kerne. Nun unterscheiden wir eine verzögerte Veröffentlichung von einer veralteten Lesekopie; beide können eine korrekte Phasenfolge unsichtbar machen.

**Folie** 32 / 50

## Write Back: neu im Cache, alt im SRAM

Synthetisches Beispiel, keine gemessene Laborantwort:

| Ort | Wert nach M7-Store, ohne Clean |
|---|---|
| M7-Cachekopie | 1, dirty |
| gemeinsamer SRAM | 0 |
| M4 liest dort | 0 |

Ein DMB macht aus dieser Dirty-Zeile keinen SRAM-Store.

**Notizen** 32 / 50

Write Back bedeutet, dass eine geänderte Cachezeile nicht bei jedem Store sofort vollständig in den Hintergrundspeicher zurückgeschrieben werden muss. Ein Store ist das Speichern eines Werts. Dirty bedeutet schmutzig beziehungsweise lokal verändert. SRAM ist Static Random Access Memory, Arbeitsspeicher. Die Zahlen null und eins sind ein synthetisches Sichtbarkeitsbeispiel und nicht die gesamte Ping-Pong-Folge.
Der M7 schreibt eins in seine Cachekopie. Solange diese Änderung nicht zurückgeschrieben wurde, kann der gemeinsam erreichbare SRAM null enthalten. Der M4 sieht dann weiterhin null. Clean ist eine Cachewartungsoperation, die geänderte Daten zurückschreibt; sie ist hier ausdrücklich nicht ausgeführt. DMB bedeutet Data Memory Barrier, Datenspeicherbarriere, und ordnet relevante Zugriffe. Sie führt dieses Clean nicht automatisch aus. Ein Cache kann Zeilen auch später verdrängen und damit den Fehler zeitweise verdecken. Ein einzelner erfolgreicher Lauf ohne passenden Cachevertrag beweist deshalb keine allgemeine Korrektheit. Im nächsten Fall hat die Gegenstelle bereits neue SRAM-Daten erzeugt, aber die M7-Lesekopie bleibt alt.

**Folie** 33 / 50

## Fremder Store: neu im SRAM, alt im M7-Cache

Synthetisches Beispiel, diesmal mit alter sauberer M7-Kopie:

| Ort | Wert nach M4-Store |
|---|---|
| gemeinsamer SRAM | 1 |
| gültige M7-Cachekopie | 0 |
| M7 liest aus seiner Kopie | 0 |

Invalidate verwirft die lokale Kopie; Dirty-Daten brauchen einen anderen Vertrag.

**Notizen** 33 / 50

Der Cortex-M4 schreibt in diesem Beispiel eins in den gemeinsamen SRAM, Static Random Access Memory oder Arbeitsspeicher. Der Cortex-M7 hat jedoch eine gültige alte Daten-Cachekopie mit null. Sein erneuter Load, das Laden des Werts, kann daraus weiterhin null erhalten. Die Kopie ist hier ausdrücklich sauber, also nicht mit eigenen unveröffentlichten Änderungen versehen.
Invalidate ist eine Cachewartungsoperation, die eine lokale Kopie verwirft. Bei passender Reihenfolge kann ein späterer Load wieder den aktuellen Hintergrundspeicher heranziehen. Das ist ein anderer Vorgang als Clean, das geänderte Daten zurückschreibt. Eine schmutzige beziehungsweise Dirty-Kopie einfach zu invalidieren kann eigene Daten verlieren. DMB, Data Memory Barrier oder Datenspeicherbarriere, erneuert die Kopie ebenfalls nicht. Die Gegenrichtung aus der vorigen Folie und dieser Lesefehler sind unabhängig zu prüfen. Für die Einsteigerbasis vermeiden wir beide durch nicht cachebare Attribute des kleinen gemeinsamen Kanals. Warum manuelles Wartungsverhalten zusätzlich Besitz ganzer Zeilen braucht, zeigt der folgende Gegenfall.

**Folie** 34 / 50

### Clean kann neuere fremde Nachbarbytes überschreiben

Eine 32-Byte-Zeile enthält zwei verschiedene Wörter A und B.

| Schritt | M7-Kopie: A/B | SRAM: A/B |
|---|---|---|
| beide zunächst null | 0 / 0 | 0 / 0 |
| M7 schreibt A | 1 / 0, dirty | 0 / 0 |
| M4 schreibt B | 1 / 0, dirty | 0 / 9 |
| M7 cleant die alte Zeile | 1 / 0 | 1 / 0 |

Getrennte Felder sind noch keine getrennten Cachezeilen.

**Notizen** 34 / 50

A und B sind zwei verschiedene Wörter innerhalb derselben 32-Byte-Daten-Cachezeile. SRAM ist der gemeinsame Static Random Access Memory, Arbeitsspeicher. Dirty bedeutet lokal verändert. Clean schreibt eine geänderte Zeile zurück. Die Tabelle ist ein absichtlich fehlerhafter Wartungsfall ohne passenden Zeilenbesitz, nicht das Verhalten der nicht cachebaren Basis.
Anfangs enthalten beide Wörter null. Der M7 ändert nur A auf eins in seiner Kopie; damit ist die Zeile schmutzig. Der M4 schreibt unabhängig neun in B des SRAM. Die alte M7-Kopie enthält für B weiterhin null. Wenn der M7 nun seine ganze alte Dirty-Zeile zurückschreibt, kann er den neueren M4-Wert neun mit null überschreiben. Jeder Kern hat zwar ein anderes logisches Wort verändert, aber die Wartung wirkt auf eine größere Einheit. Auch ein gemischter Kontroll-/Payloadblock kann solche Schäden tragen. Eine brauchbare Wartungsvariante benötigt daher exklusive Zuständigkeit für die betroffenen vollständigen Zeilen und eine geprüfte Übergabefolge. Feld-Ownership und Cachezeilen-Ownership sind verschiedene Granularitäten.

**Folie** 35 / 50

### Zeilengrenzen nachrechnen statt nur den Pointer ausrichten

Lehradresse `0x38000014`, Länge 16 Byte, Zeile 32 Byte.

$$a_0 = \left\lfloor \frac{a}{32} \right\rfloor \cdot 32, \quad e = \left\lceil \frac{a+L}{32} \right\rceil \cdot 32$$

- Beginn: `0x38000000`
- exklusives Ende: `0x38000040`
- Wartungsspanne: 64 Byte, zwei Zeilen
- Nachbarobjekte in dieser Spanne brauchen denselben Besitzvertrag

**Notizen** 35 / 50

a ist die Startadresse als Zahl, L die Nutzlänge in Byte, a null der abgerundete Zeilenbeginn und e das aufgerundete exklusive Ende. Die Klammern floor und ceil bedeuten Abrunden und Aufrunden. Die Zeilengröße beträgt beim betrachteten Cortex-M7 32 Byte. Das Beispiel verwendet eine didaktisch vorgegebene Adresse und ist kein tatsächliches Labormessobjekt.
0x14 ist der hexadezimale Offset zwanzig innerhalb einer Zeile. Sechzehn weitere Byte reichen bis zum exklusiven Offset 0x24, also sechsunddreißig. Damit werden die erste Zeile null bis einunddreißig und die zweite zweiunddreißig bis dreiundsechzig berührt. Der Beginn der Wartung ist 0x38000000, das exklusive Ende 0x38000040. Die Differenz beträgt 64 Byte. Eine allein auf 32 Byte ausgerichtete Startadresse würde außerdem keine beliebige Länge passend machen. Cachewartung darf keine fremden Dirty-Daten in diesen Zeilen unabsichtlich überschreiben oder verwerfen. Auch die Zugriffsgrenzen und der Vertrag der eingebundenen Wartungsfunktion werden geprüft, bevor ein solcher Bereichscode verwendet wird.

**Folie** 36 / 50

## Cachewartung ist ein vollständiges Übergabeprotokoll

- Schreiber: Daten fertigstellen, passende Zeilen cleanen, dann veröffentlichen
- Leser: eigene zulässige Kopien invalidieren, dann konsumieren
- Kontrollinformationen brauchen ebenfalls eine sichere Sicht
- Ganze Zeilen müssen gültige Zuständigkeit haben

CMSIS-M7-Funktionen: `SCB_CleanDCache_by_Addr`, `SCB_InvalidateDCache_by_Addr`.

**Notizen** 36 / 50

CMSIS bedeutet Common Microcontroller Software Interface Standard, die standardisierte Cortex-M-Unterstützung. SCB bezeichnet den System Control Block, den Systemsteuerblock. Die genannten C-Funktionen sind adressbezogene Daten-Cachewartung des M7. Clean schreibt Änderungen zurück; Invalidate verwirft Kopien. Ihre konkreten Signaturen, Grenzforderungen und enthaltenen Barrieren stehen im eingebundenen Header, nicht in einer beliebig neueren Onlineversion.
Die Stichpunkte benennen das Entwurfsmuster, keinen vollständigen Wartungsalgorithmus für unseren gemischten Block. Ein Schreiber muss die Daten fertigstellen und ihre passenden Zeilen vor der Veröffentlichung sichtbar machen. Ein Leser darf nur Kopien verwerfen, deren Verlust keine eigenen unveröffentlichten Daten zerstört. Auch das Kontrollsignal darf nicht in einer unbemerkt alten Cachekopie verbleiben. Getrennte Payloadzeilen allein lösen das Problem eines cachebaren Zustandsworts nicht. Die Start- und Endrundung aus der vorigen Folie sowie exklusiver Zeilenbesitz gehören dazu. Die Pflichtübung implementiert diese komplexere Variante deshalb nicht zusätzlich, sondern verwendet die klar abgegrenzte nicht cachebare Basis und einen kontrollierten Fehlerbuild.

**Folie** 37 / 50

### MPU: nicht cachebare Attribute für den Lehrkanal

M7-Basisregion: `0x38000000`, 64 KiB.

| Eigenschaft | Gewählte Basis |
|---|---|
| Datenzugriff | Shared-Region nicht cachebar |
| Rechte | Lesen/Schreiben, keine Ausführung |
| Geltung | Zugriffe des M7 |
| Einrichtung | vor Cache und gemeinsamer Nutzung |

Andere M7-Regionen können weiter den Cache nutzen.

**Notizen** 37 / 50

MPU bedeutet Memory Protection Unit, Speicherschutzeinheit. Sie beschreibt Regionen des Cortex-M7 mit Zugriffsrechten und Speicherattributen. KiB bedeutet Kibibyte mit 1024 Byte; die Basis umfasst SRAM4 ab 0x38000000 mit 64 KiB. Nicht cachebar verhindert für diese Zugriffe die private Daten-Cachekopie des M7. Der gewählte normale Speichertyp erlaubt die vorgesehenen Datenzugriffe. Das Ausführen von Instruktionen aus dieser Datenregion ist gesperrt; die entsprechende Eigenschaft heißt Execute Never. Die konkrete Registerkodierung und der vollständige Konfigurationshelfer stehen in der Implementierungshilfe. Dort ist TEX die Type Extension, Attributerweiterung, C bedeutet cacheable, B bufferable und S shareable. Für die Basis lauten die Werte TEX eins, C null, B null und S eins. Shareable erzeugt auf diesem System keinen automatischen M7/M4-Cacheabgleich. Die Einrichtung erfolgt beim gemeinsamen Neustart vor Daten-Cache-Aktivierung und Kommunikation. Andere Regionen können weiterhin vom Cache profitieren. Der bereitgestellte Helfer verwendet HAL-Konstanten; HAL ist die Hardware Abstraction Layer, Hardwareabstraktion. Für die Beurteilung bleibt wesentlich, welche effektiven Rechte und Attribute der reale Zugriff hat, auch wenn mehrere konfigurierte Regionen überlappen.

**Folie** 38 / 50

### Bei Überlappung entscheidet die höher nummerierte Region

Lehrbeispiel für dieselbe Adresse:

| Region | Bereich | Attribut |
|---|---|---|
| 4 | erste 64 KiB SRAM4 | nicht cachebar |
| 6 | erste 32 KiB SRAM4 | cachebar, S = 0 |

Für `ipc` am Anfang gewinnt Region 6.
Region aktiviert, Subregionen und effektive Registerwerte prüfen.

**Notizen** 38 / 50

Das Beispiel verwendet die Cortex-M-MPU, Memory Protection Unit oder Speicherschutzeinheit, des betrachteten Armv7-M-Systems. SRAM4 ist die gemeinsame Arbeitsspeicherbank. KiB bedeutet Kibibyte. Die gezeigten Regionen sind aktiviert und ihre betreffenden Subregionen, Unterbereiche, nicht deaktiviert. Für eine Adresse, die beide Regionen trifft, gewinnt die passende Region mit der höheren Nummer.
ipc liegt am Beginn 0x38000000 und damit in beiden Bereichen. Region sechs macht diesen Zugriff im Beispiel cachebar, obwohl Region vier ihn nicht cachebar beschreiben würde. S bezeichnet das Shareable-Attribut; das Beispiel der cachebaren Region verwendet S null. Ein korrekt aussehender Aufruf für die Basisregion reicht deshalb nicht, wenn eine spätere oder höher priorisierte Region ihre Absicht aufhebt. Der Regionplatz wird im Projekt bewusst reserviert. Der M7-Helfer prüft dessen numerische Gültigkeit und die ausgeschaltete Daten-Cache-Voraussetzung, erkennt aber nicht jede Überlappung. Das Registerlesen und die vollständige regionale Projektprüfung bleiben notwendige Integrationsschritte. Papieraufgabe 4 verwendet genau dieses Gegenbeispiel.

**Folie** 39 / 50

## MPU-Grenzen sind kernlokale Grenzen

- M7-MPU beschreibt und begrenzt M7-Zugriffe
- Sie schützt nicht automatisch M4 oder DMA
- Nicht cachebar bedeutet nicht konfliktfrei
- Magic ersetzt kein Reset- oder Fehlerprotokoll
- Schreibrechte benötigen weiterhin die Phasenregel

**Notizen** 39 / 50

MPU steht für Memory Protection Unit, Speicherschutzeinheit. DMA bedeutet Direct Memory Access, direkter Hardware-Speicherzugriff. Die MPU des Cortex-M7 ist nicht eine zentrale Schutzverwaltung sämtlicher Zugriffe des Chips. Ihre Einstellungen kontrollieren die entsprechenden M7-Zugriffe und deren Attribute. Der Cortex-M4 und weitere Hardwaremaster haben dadurch nicht automatisch denselben Schutzvertrag.
Nicht cachebar verhindert in der Basis eine bestimmte veraltete Datenkopie. Beide Kerne könnten dennoch dieselben Bytes gleichzeitig falsch verändern. Das bekannte Read-Modify-Write, Lesen, Ändern und Zurückschreiben, braucht weiterhin einen Besitzer oder eine passende Sperre. Magic ist unsere Kennung im Datenblock und keine automatische Sitzungsnummer, die jeden einseitigen Neustart erkennt. Es ist deshalb kein allgemeiner Recovery-Nachweis; Recovery bedeutet Wiederaufnahme nach einem Fehler. Die Basis stoppt bei einem ungültigen Zustand oder Zeitablauf und setzt den Block nicht während fremder Nutzung zurück. Diese Grenzen sind ein bewusster einfacher Vertrag. Weitergehende Schutz- und Wiederanlaufmechanismen folgen später im Kurs.

**Folie** 40 / 50

## Optionale Fehlervariante: eigener Neustart

| M7-Attribut | Basis | Fehlerbuild |
|---|---|---|
| M7-Zugriff | nicht cachebar | cachebar, Write Back |
| Region | SRAM4, 64 KiB | gleiche Region |
| D-Cache | darf an sein | muss tatsächlich an sein |
| Wartung | für diesen Kanal nicht nötig | bewusst nicht eingebaut |

Mögliche Folge: alter Wert oder Timeout; kein garantierter Stillstand.

**Notizen** 40 / 50

TEX ist Type Extension, C cacheable, B bufferable und S shareable; diese Felder bestimmen den effektiven M7-Speichertyp. SRAM4 ist der gemeinsame Arbeitsspeicher, KiB ein Kibibyte. D-Cache ist der Daten-Cache. Der Fehlerbuild wählt einen cachebaren Normal-Write-Back-Typ mit S null und aktiviert den Daten-Cache. Write Back bedeutet spätere Rückschreibung geänderter Kopien. S eins würde auf dem M7 nicht einfach einen Snooping-Nachweis herstellen und ist keine sichere isolierte Variation nur eines C-Bits. Die Initialdaten werden in unserem Ablauf bei noch ausgeschaltetem D-Cache in SRAM geschrieben. Anschließend können eine alte M7-Lesekopie oder eine neue nur lokale Anfrage die Laufzeitsicht zerstören. Cacheverdrängung, Debugbeobachtung und konkreter Ablauf können den Fehler verändern oder verdecken. Erwartet wird darum kein erfundener immer gleicher Timeout. Attribute werden ausschließlich beim gemeinsamen Neustart geändert. Der Basisbuild bleibt der gültige Kanal, die Fehlervariante eine absichtliche Diagnoseübung. Aus ihrem Verlauf entsteht kein behaupteter erfolgreich gepflegter Write-Back-Kommunikationskanal.

**Folie** 41 / 50

## Timeout bedeutet Diagnose, nicht Besitzübernahme

Im Modul getrennte private Fehlercodes:

- Setup oder frühe Cacheaktivierung
- ungültige Kennung oder Protokollphase
- unerwarteter Payload-Wert
- Ablauf des lokalen Wartebudgets

Kein `memset` des Shared-Blocks als spontane Reparatur.

**Notizen** 41 / 50

Timeout ist der Ablauf eines Wartebudgets. Payload ist der Nutzwert. memset ist die C-Funktion zum Füllen eines Speicherbereichs. Ein solcher Reset des gemeinsamen Blocks könnte eine noch aktive Gegenstelle überschreiben und ist daher kein zulässiger automatischer Fehlerpfad.

**Notizen, Fortsetzung** 41 / 50

Die vollständige Anwendung führt lokale Diagnosevariablen außerhalb der gemeinsamen Sektion. Ein Fehler stoppt diesen Anwendungspfad. Der M7 nutzt eine laufende HAL-Millisekundenbasis, Hardware Abstraction Layer oder Hardwareabstraktion, und ein Budget von 2000 Millisekunden. Der M4 nutzt 2000 Kernel-Ticks, bei bestätigter 1000-Hertz-Basis nominell etwa zwei Sekunden. Hertz zählt Ereignisse pro Sekunde. Prüfung erfolgt erst, wenn der jeweilige Code wieder ausgeführt wird. Eine angehaltene Zeitbasis oder ein dauerhaft nicht zugeteilter Worker wird damit nicht unabhängig überwacht. Der Timeout ist also kein Watchdog und keine bewiesene IPC-Deadline. IPC ist Inter-Processor Communication, Kommunikation zwischen Kernen. Nach einem Fehler werden Start, Adressen, Attribute und Protokoll geprüft und beide Kerne für einen neuen Basislauf gemeinsam gestartet.

**Folie** 42 / 50

## Antwortwerte und Latenzen sind verschiedene Nachweise

- Private Antworten 1, 3, 5 belegen den beobachteten Datenlauf
- `osDelay(1)` wartet einen Kernel-Tick
- `HAL_Delay(1)` ist eine lokale HAL-Wartefunktion
- Debughalt und UART-Ausgabe verändern den Ablauf
- Zwei lokale Zykluszähler brauchen erst einen gemeinsamen Zeitbezug

$$R = f - r \quad \text{mit ausdrücklich definiertem Ereignis und Abschluss}$$

**Notizen** 42 / 50

R ist die Antwortzeit, r die definierte Freigabe oder das Ausgangsereignis und f der geforderte Abschluss. Die Gleichung ist die bekannte Zeitdefinition, keine Messung. Die Werte eins, drei und fünf sind Nutzdaten, nicht Millisekunden oder Latenzen. osDelay ist der CMSIS-RTOS2-Warteaufruf; CMSIS bedeutet Common Microcontroller Software Interface Standard. Eine Wartezahl eins zählt einen Kernel-Tick. HAL bedeutet Hardware Abstraction Layer, Hardwareabstraktion. HAL_Delay ist im M7-Bare-Metal-Programm eine lokale aktive Wartefunktion und keine RTOS-Blockierung.
UART ist Universal Asynchronous Receiver/Transmitter, serielle Schnittstelle. Eine Ausgabe kann Formatierungs-, Geräte- und Wartekosten hinzufügen. Ein Debughalt verändert außerdem die relative Ausführung beider Kerne. Zykluszähler zählen lokale Prozessortakte und sind ohne bestätigte Skalierung und Synchronisation keine gemeinsame Uhr. Für eine echte IPC-Latenz, Inter-Processor Communication oder Kommunikation zwischen Kernen, müssen Anfang, Ende und Zeitbasis definiert sein. Eine erste Antwort kann den Boot- und Kernelstart einschließen. Im Basisversuch wird deshalb eine Datenfolge geprüft; optionales Timing erhält einen eigenen Messvertrag.

**Folie** 43 / 50

### Laborbasis: zwei Ziele, ein gemeinsamer Header

Anwendungsdateien unter `code/lab07/`:

- `lab07_ipc.h` und `lab07_shared.c` in beiden Images
- `lab07_cm7.c` nur auf CM7
- `lab07_cm4.c` nur auf CM4
- `shared-section.ld` als Linker-Integrationsfragment

CubeMX und VS Code mit STM32 VS Code Extension.

**Notizen** 43 / 50

CM7 und CM4 sind die beiden Kernziele für Cortex-M7 und Cortex-M4. Ein Image ist das übersetzte Programm eines Kerns. STM32CubeMX erzeugt die Projektbasis, Visual Studio Code mit STM32 VS Code Extension baut und untersucht die CMake-Ziele. CMake beschreibt den Build. Die genannten Dateien existieren als vollständige Anwendungsreferenz im Kursrepository; sie sind kein fertig generiertes Firmwareprojekt.
lab07_ipc.h enthält Layout, Konstanten und Deklarationen. lab07_shared.c definiert das gemeinsame Objekt und wird genau einmal in jedes Ziel aufgenommen. Die beiden Kernmodule gehören jeweils nur in ihr Ziel. Die vorhandenen Buildkennungen CORE_CM7 beziehungsweise CORE_CM4 müssen stimmen. shared-section.ld ist bewusst nur ein Integrationsfragment: Es ersetzt nicht blind die vollständige erzeugte Linkerdatei. Private globale Diagnosevariablen bleiben in den privaten Sektionen der Images. Die Lehrperson stellt für diesen Versuch eine geprüft lauffähige Projektbasis mit Dual-Core-Boot und M4-Kernel bereit. Seine bisherigen Anwendungslasten und Callbackexperimente werden für diesen isolierten Versuch deaktiviert.

**Folie** 44 / 50

### Cacheaktivierung nicht vor die Shared-Initialisierung legen

M7-Reihenfolge im vorbereiteten Projekt:

1. Projekt-MPU und freie Shared-Region konfigurieren
2. M4-Stop sowie HAL- und Taktstart wie vereinbart
3. `Lab07_CM7_InitShared()` bei D-Cache aus
4. D-Cache einschalten, Boot-HSEM freigeben
5. `Lab07_CM7_Run()` einmal ausführen

Frühe Cachehelper und spätere MPUhelper auf Doppelkonfiguration prüfen.

**Notizen** 44 / 50

MPU bedeutet Memory Protection Unit, Speicherschutzeinheit. HAL ist Hardware Abstraction Layer, Hardwareabstraktion. HSEM ist der Hardware Semaphore, hier das Bootsignal. D-Cache bezeichnet den Daten-Cache. Viele Vorlagen aktivieren in einem frühen CPU_CACHE_Enable-Helfer sowohl Instruktions-

**Notizen, Fortsetzung** 44 / 50

als auch Daten-Cache. Für unser kontrolliertes Labor wird die D-Cache-Aktivierung einmal nach die Shared-Initialisierung gelegt. Die Instruktions-Cache-Entscheidung kann getrennt bleiben.
Die Referenzfunktion Lab07_CM7_InitShared verlangt den ausgeschalteten Daten-Cache und die bereits konfigurierte Region. Sie wird vor dem vorhandenen HSEM-Release eingefügt, nachdem die benötigten Takte bereitstehen. Wird der Cache unbemerkt vorher aktiviert oder die Region nachträglich überschrieben, stoppt der Helfer beziehungsweise die effektive Attributprüfung deckt den Integrationsfehler auf. Das erzeugte Boot-Warten bleibt erhalten. Der M4 initialisiert den Shared-Block nach der Freigabe nicht erneut. Die konkrete Stelle ist am lokalen Code zu prüfen, weil ein allgemeiner USER-CODE-Block nach allen Peripherieinitialisierungen bereits zu spät sein kann. Die Reihenfolge verbindet Bootvertrag und Cacheexperiment ohne laufende Attributumschaltung.

**Folie** 45 / 50

## Ein M4-Worker übernimmt drei Antworten

- CMSIS_V2 / FreeRTOS auf CM4
- Ein Worker mit zunächst 1024 Byte Stack
- Aufruf: `Lab07_CM4_Run(argument)`
- Nur erwartete Phase erlaubt das Lesen und Ändern
- Nach drei Antworten beendet sich der Thread

Vorhandenen Kernelstart und Bootpfad einmal behalten.

**Notizen** 45 / 50

CMSIS_V2 ist die Cube-Auswahl der zweiten Betriebssystem-Schnittstelle des Common Microcontroller Software Interface Standard. FreeRTOS ist der M4-Kernel. Ein Worker ist ein verarbeitender Thread. Der vorgeschlagene Stack mit 1024 Byte ist ein zu prüfender Startwert, keine nachgewiesene größte Speicherbelegung.
Die erzeugte Worker-Einstiegsfunktion ruft Lab07_CM4_Run mit ihrem vorhandenen Argument auf. Es wird keine zweite gleichnamige Eintrittsfunktion angelegt. Kernelinitialisierung, Threadanlage und Kernelstart existieren bereits in der Basis. Das gültige Threadhandle wird geprüft. Während RESPONSE_M7 wartet der Worker bei Bedarf mit osDelay einen Tick. Bei REQUEST_M4 prüft er nach der Barriere die erwartete Anfragezahl, erhöht sie und veröffentlicht die Antwort. Nach der dritten Antwort markiert er seine private Diagnose und beendet sich über osThreadExit, den CMSIS-Threadbeendigungsaufruf. Eine später automatisch wiederholte Aktivierung gehört nicht zum endlichen Versuch. Frühaufruf, falsche Kennung, unzulässige Phase und technische Wartefehler sind eigene Fehlerpfade und keine gewöhnlichen Antworten.

**Folie** 46 / 50

## Was der erfolgreiche Basislauf tatsächlich zeigen muss

| Prüfstelle | Erwartetes Ergebnis |
|---|---|
| zwei Maps | `ipc` an derselben festen Adresse |
| effektive M7-MPU | Shared-Region nicht cachebar |
| CM7-Diagnose | Antworten 1, 3, 5; Done = 1 |
| CM4-Diagnose | drei Antworten; Done = 1 |
| Shared-Ende | Payload 5, Zustand TURN_M7 |

Alle Kernzustände gemeinsam anhalten; Hardwarezeiten getrennt protokollieren.

**Notizen** 46 / 50

Eine Map ist das Platzierungsprotokoll des Linkers. MPU bedeutet Memory Protection Unit, Speicherschutzeinheit. Die Tabelle beschreibt die Kontrollgrößen des Basislaufs und ist kein erfundenes Ergebnis auf einem bereits getesteten Board. ipc ist das gemeinsame Objekt. Done ist ein privates Fertigkennzeichen, das jeweilige Kernmodul schreibt es nach seinem vorgesehenen Ablauf.
Der M7 muss drei Antworten mit eins, drei und fünf protokolliert haben. Der M4 hat drei Antworten erzeugt und beendet seinen Worker. Der gemeinsame Nutzwert bleibt fünf, der Zustand TURN_M7. Diese Endwerte unterscheiden den endlichen Versuch von einem Kanal, der fortlaufend neue Requests sendet. Erst wenn beide Kerne zur passenden Zeit angehalten sind, können ihre Diagnosefelder und der Shared-Block als zusammengehöriger Snapshot, also Momentaufnahme, gelesen werden. Ein Debugfenster kann eine andere Speicheransicht zeigen als ein laufender M7-Load; es ist kein automatischer Cachebeweis. Ein gelinktes Objekt und ein erfolgreicher Datenlauf sind wiederum andere Nachweise als eine gehaltene Mikrosekundenfrist.

**Folie** 47 / 50

## Optionaler Fehlerbuild: beobachten und Basis herstellen

Nur den M7-Modus auf `LAB07_CACHED_FAULT` ändern.

- gleicher Block, gleiche Phasenfolge, gemeinsamer Reset
- D-Cache und effektive WB-Attribute kontrollieren
- Datenfolge oder Timeout tatsächlich notieren
- keine spontane Cachewartung am gemischten Block
- anschließend `LAB07_NONCACHED` wiederherstellen

**Notizen** 47 / 50

LAB07_CACHED_FAULT ist der absichtliche Fehler-Modus der Referenz, LAB07_NONCACHED die gültige

**Notizen, Fortsetzung** 47 / 50

nicht cachebare Basis. WB bedeutet Write Back, verzögerte Rückschreibung geänderter Cachekopien. D-Cache ist der Daten-Cache. Der M7-Modus wird beim gemeinsamen Neustart geändert, nicht in einem laufenden Debugdialog.
Unverändert bleiben Layout, Phasenfolge und M4-Code. So kann ein beobachteter Unterschied sinnvoll mit der veränderten M7-Sicht in Beziehung gesetzt werden. Zunächst werden tatsächliche Cacheaktivierung, MPU-Attribute und Überlappung geprüft; MPU bedeutet Memory Protection Unit, Speicherschutzeinheit. Ein Timeout oder ein alter Wert kann auftreten. Ein erfolgreicher Lauf durch Verdrängung oder spezielle Startreihenfolge beweist dagegen keinen sicheren Write-Back-Kanal. Die Reaktion auf den Fehler ist Diagnose und Rückkehr zur geprüften Basis. Ein pauschales Clean oder Invalidate des gemeinsam von beiden beschriebenen Blocks wäre gerade der ungeprüfte Reparaturversuch, vor dem das Zeilenbeispiel warnt. Die praktische Beobachtung wird dokumentiert und nicht durch erwartete Zahlen ersetzt.

**Folie** 48 / 50

## Papierprüfung: passende Mittel mit ihren Grenzen

1. Queue-Handle im Shared SRAM: gemeinsamer Kanal?
2. Payload vor Zustand: fehlt noch ein Cachevertrag?
3. Zwölf Typbytes: wie viel Zeilenreserve?
4. Höher nummerierte MPU-Überlappung: welches Attribut gilt?

Begründen Sie jede Antwort, bevor Sie den Versuch bewerten.

**Notizen** 48 / 50

Eine Queue ist eine begrenzte Nachrichtenwarteschlange; ihr Handle ist hier ein lokaler Kernelbezeichner. Shared SRAM bedeutet gemeinsam erreichbaren Static Random Access Memory, Arbeitsspeicher. Den Handle dort zu speichern macht die M7-Firmware nicht zum Teilnehmer der M4-Verwaltung. Der erste Fall braucht einen eigenen Kanalvertrag.
Payload ist der Nutzwert. Eine korrekte Veröffentlichung vor dem Zustandswechsel ordnet die Nachricht, ersetzt aber keine Sichtbarkeitsregel für Cachekopien. Zwölf Typbytes passen im Referenzobjekt in eine auf 32 Byte ausgerichtete und reservierte Zeile. Der größere MPU-Bereich mit 64 KiB, also 65536 Byte, ist nochmals eine andere Größe. MPU bedeutet Memory Protection Unit, Speicherschutzeinheit. Bei einer passenden Überlappung gewinnt die höher nummerierte aktivierte Region unter den genannten Armv7-M-Annahmen. Diese vier Antworten verbinden die Klausurfragen mit der Integrationsprüfung. Wer nur einen erfolgreichen Zahlenlauf betrachtet, kann die fehlenden Voraussetzungen leicht übersehen. Der nächste Rückblick ordnet die Begriffe nochmals nach ihrem jeweiligen Problem.

**Folie** 49 / 50

## Quiz und Anschluss an die nächsten Termine

- AMP trennt Softwareverwaltung, nicht alle Störquellen
- Gleiche Adresse ist nötig, Besitz und Sichtbarkeit ebenso
- Barrieren und Cachewartung lösen verschiedene Probleme
- MPU-Attribute wirken lokal und nach Regionspriorität
- Zehn Anwendungsfragen statt reiner Begriffsabfrage

Vorlesung 8: Messungen bewerten · Vorlesung 9: Verträge zusammenführen.

**Notizen** 49 / 50

AMP ist Asymmetric Multiprocessing, getrennte Mehrkernsoftware. Die Architektur erleichtert Zuständigkeiten, erzeugt aber keine vollständige Hardwareisolation. Die gleiche Datenadresse ist eine notwendige Kanalvoraussetzung. Besitz sagt, wer schreiben darf; Sichtbarkeit sagt, welche Bytes ein Zugriff tatsächlich liefert. Eine Barriere ordnet relevante Zugriffe, Cachewartung behandelt lokale Kopien. MPU ist die Memory Protection Unit, Speicherschutzeinheit, deren effektive Region und lokale Reichweite geprüft werden.

Das Quiz mit zehn Fragen prüft diese Unterscheidungen an Fällen aus der Vorlesung. Begründen Sie beispielsweise, warum ein M4-Mutex den M7 nicht automatisch schützt oder warum zwölf Bytes typisierte Nutzstruktur und 32 reservierte Bytes kein Widerspruch sind. Die nächste Vorlesung betrachtet die Beobachtung selbst: Ein protokollierter Kernzugriff ist nicht automatisch der aktuelle SRAM-Wert einer Gegenstelle. Vorlesung 6 hat bereits gezeigt, wie Budgets für eigene Prozessorzeit die Ereignisarbeit begrenzen. Die Fallstudie in Vorlesung 9 verbindet diese Zeitregeln mit Datenbesitz und Fortschrittsüberwachung. Die weiterführenden HSEM-Laufzeitversuche bleiben als optionale Vertiefung unter code/archiv/multicore-safety verfügbar.

**Folie** 50 / 50

## Quellen und Nachweise auseinanderhalten

- Buttazzo: *Hard Real-Time Computing Systems*, 3. Auflage
- ST: RM0399, UM2408, AN5617, AN4838 und AN5361
- ST-Dual-Core-Boottemplate
- STM32H7-HAL: MPU
- CMSIS-M7: Cachewartung

**Notizen** 50 / 50

Giorgio C. Buttazzos Hard Real-Time Computing Systems ist die zentrale Literatur für Zeitmodelle und Ressourcenabhängigkeiten. STs RM0399 ist das Referenzhandbuch der betreffenden Mikrocontrollerfamilie, UM2408 das Benutzerhandbuch der Boardfamilie. AN5617 behandelt Mehrkernkommunikation, AN4838 MPU-Attribute und AN5361 den Dual-Core-Entwicklungsablauf. MPU bedeutet Memory Protection

**Notizen, Fortsetzung** 50 / 50

Unit, Speicherschutzeinheit.
Das eingesehene Boottemplate gehört zum H745-Beispiel und belegt das grundsätzliche Stop-/HSEM-Muster; es ersetzt nicht die konkrete H755-Projektprüfung. HSEM ist Hardware Semaphore. HAL bedeutet Hardware Abstraction Layer, Hardwareabstraktion. Der tatsächlich eingesehene HAL-Header enthält die verwendeten Regionsattribute, der CMSIS-Header die Cachewartung. CMSIS steht für Common Microcontroller Software Interface Standard. Öffentliche Entwicklungszweige sind veränderlich; die eingebundenen Projektversionen werden protokolliert. Eigene Zahlen sind als Modellwerte gekennzeichnet. Syntaxprüfung, simuliertes Protokoll, Linkerprüfung, Renderprüfung, tatsächlicher STM32-Build und Boardmessung sind unterschiedliche Evidenz. Ohne vorhandenes Zielprojekt und Gerät wird kein Hardwareerfolg behauptet.